Stigmergic Project Delivery: Dynamic Specification-Driven Planning and Trace-Based Forecasting

Matthew Fyfe • September 2026

Abstract

Project planning and delivery rely on manual process design that decouples from execution and gives forecasts ungrounded in resource-level execution data. Though these areas are each well studied, to our knowledge no single model derives process directly from a specification while also forecasting outcomes and allocating both work and resources. We propose Stigmergic Project Delivery (SPD), a formal model in which a specification generates process structure and remains dynamic during execution. We show how traces left in the process graph during execution drive both self-selected allocation across agentic resources and trace-based forecasting, without central control. We explain how SPD instantiates Heylighen's components of stigmergy, with coordination emerging from them, and compare it against product-based workflow design, Activity-on-Node scheduling and predictive process monitoring to identify what each leaves unresolved. By deriving both risk-aware forecasts and task allocation directly from the specification and trace, SPD is intended to improve the predictability of delivery and reduce the forecasting and coordination overhead carried by management layers.

1. Introduction

Project planning methods typically handle three things separately that ought to inform one another. They derive process structure by hand rather than from the product to be built; they forecast performance from assumed or generic distributions, or from project-level history, rather than from a recorded history of the agents, tools, and materials involved; and they coordinate who does the work through central assignment, rather than letting it fall out of the work's own structure, in which the skillsets needed and the relevant experience are already defined by the derivation.

This paper's contribution is Stigmergic Project Delivery (SPD), a single model in which process derivation draws on the specification, and trace-based forecasting and self-selected task allocation both draw on the trace that execution accumulates.

In SPD, a master specification holding all variation in the product and a master process holding all variation in its delivery are initialised. Each specification node is then linked to the steps it necessitates, so a selection within the product determines the process that delivers it. Within the process, the steps are joined by directed edges, each carrying a product that one step produces and the next consumes; these products are the inputs a step takes and the outputs it must produce, and so form its contract. Steps are grouped by skillset, and specification nodes may carry descriptors, values that scale the outcomes recorded against the steps they necessitate.

Each project begins with a provided instantiation of the specification, a subset of the master with its descriptors bound, from which the corresponding subset of the master process is derived. The order of steps follows from the products the master process routes between them, authored once and inherited rather than hand-specified per project, as Activity-on-Node scheduling and BPMN require. The specification and its derived process remain dynamic and extensible during execution. From the trace of earlier projects, each derived step carries a forecast of its delivery metrics, material usage, time, and whatever else the domain records; the same trace fixes which agents are eligible for it. Eligible agents self-select to take up each skillset's contract, and as they work, data on those metrics accrues back into the trace, manually or via telemetry. The trace persists across projects, closing the loop: each project forecasts and allocates from the history before it, and leaves its own for the next.

Section 2 positions SPD against prior work along the three legs it combines. Section 3 gives the formal model: the specification, the mapping, the trace, the eligibility and self-selection they drive, and the contract it produces. Section 4 works two concrete examples by hand. Section 5 outlines the evaluation this claim invites, since this paper introduces and positions the model rather than executing that evaluation. Section 6 discusses the model's limitations and where it does not apply. Section 7 concludes.

2. Related work

We position SPD against prior work on two fronts: the three legs it combines, deriving process from specification, forecasting from a trace, and coordinating allocation through that same trace; and the two formalisms it extends, the activity-on-node process graph and the resource model.

Process derivation from specification. The idea that process structure should follow from product structure, rather than being authored independently, originates with van der Aalst [1] and was developed into product-based workflow design (PBWD) by Reijers, Limam, and van der Aalst [2], who derive a workflow from a product specification and design criteria using formal methods. PBWD's product data model fixes a single cost, time, and probability per production rule at design time, and cannot represent repeated components. SPD addresses both issues, building on the and/or feature tree of Kang et al. [3] and its later cardinality-based extensions [4].

Rolling specification and continuous derivation. A further limitation of PBWD is temporal rather than structural: it derives a workflow once, from a fixed specification, and does not accommodate a specification that changes after execution begins. Case handling [5] takes the opposite stance, deriving what can be done from case data as it arrives rather than from a pre-specified control flow, but PBWD's own authors found the two could not be bridged: current case handling could not adequately support product-based design [6]. SPD does both at once: its derivation is continuous, so an extension to the specification during execution extends the derived process with it. Section 4 documents this in full.

Network formalisms. SPD's process graph is a modified Activity-on-Node graph [7], where edges are promoted to a first class object representing the products passed between the activities. That typing is critical to SPD: it gives each step a contract, the inputs it consumes and the outputs it must produce, which is what an agent takes on and is held to, where a conventional precedence link expresses only ordering and so offers nothing to commit to. The typed product also carries its intrinsic properties, such as the time a crop needs to mature before harvest, which a downstream step reads directly rather than through the lag or dummy activity conventional AON forces a modeller to author.

Resources and agency. No existing academic resource formalism captures the three-way split SPD needs, self-directing agentic resources, tools granted for use, and materials embodied in the product: the Resource-Task Network [8, 9] models materials, equipment, and labour within a single uniform abstraction, and multi-agent systems research [10] separates autonomous agents from passive artifacts but not materials. The closest match is an industry taxonomy rather than an academic one, the six construction-resource subtypes Industry Foundation Classes [11] standardise: IfcLaborResource, IfcCrewResource, and IfcSubContractResource fold into agentic; IfcConstructionEquipmentResource is tools; and IfcConstructionMaterialResource and IfcConstructionProductResource are materials. In all of them, IFC included, labour is a resource the model allocates; SPD's extension is to treat it as an agent rather than an allocatable resource.

Trace-based forecasting. Predictive process monitoring, surveyed by Márquez-Chamorro, Resinas, and Ruiz-Cortés [12], forecasts properties of an ongoing process instance (remaining time, cost, or outcome) from historical event logs, and has been shown to benefit from resource- and workload-specific features. This establishes that forecasting from resource-attributed history is well-developed in its own right, but it forecasts within a process it assumes already exists, rather than deriving that process. Within project management

specifically, schedule risk analysis [13-15] measures activity and resource sensitivity via Monte Carlo simulation over the network, typically using distributions fitted to subjective three-point estimates [16] rather than measured performance of specific agents, tools, or materials. Vanhoucke and colleagues have moved this line from expert-estimated inputs towards measured history: machine learning trained on past projects [17] and nearest-neighbour matching against comparable ones [18] both forecast from recorded outcomes. Matching against comparable projects is reference-class forecasting [19], which predicts from the outcomes of past cases similar to the one at hand rather than from expert judgment; Batselier and Vanhoucke evaluate it for project management and integrate it with earned value management [20, 21], drawing on the empirical project database they document [22]. These methods learn at the level of the project or activity, the level their data sits at; but such data is itself hard to come by, the field often leaning on artificial data [23], and history at the granularity of individual agents, tools, and materials is scarcer still. SPD closes this gap by forming the reference class at any level the work is decomposed to, from the organisation down to the individual action, not only the whole project: the finer units recur across otherwise-unique projects, so their history accrues as a by-product of execution where sparse, rarely-repeated project history does not. The finer the level recorded, the more reducible uncertainty is removed, until at the finest level, where telemetry can record the individual action, only irreducible aleatory variation remains [24], the floor that rules out deterministic forecasting.

Coordination mechanism. Stigmergy, coordination through traces left in a shared medium rather than direct instruction, was first described by Grassé [25] and given formal treatment by Heylighen [26, 27], who defines it through five components: action, agent, medium, trace, and coordination. SPD provides the first four: an action is a step's execution; agents are the executors; the medium is the process graph; and the trace is the record that execution leaves. Stigmergic coordination emerges from these as the resulting self-selection: agents are filtered to an eligible set by matching recorded experience against each step's requirements, and the forecast attached to the step, drawn from its trace, is the stimulus an eligible agent then chooses on, rather than being centrally assigned. One of Heylighen's own illustrations of stigmergic coordination among humans, programmers self-selecting from a shared, largely independent backlog, demonstrates the same underlying principle in its simplest form; SPD extends it to a structurally derived graph of dependent steps, each bound by a contract between agent and product, rather than an unstructured or independent task. This is not the Contract Net Protocol [28], the classic contract-based scheme in multi-agent systems, where a manager announces a task, eligible agents bid, and the manager awards the contract to a chosen bid; SPD has no announcement, no bidding and no central award, an eligible agent self-selects on the trace stimulus directly.

The same coordination mechanism is attested in construction, a domain of structured projects with interdependent skillsets rather than isolated tasks: Christensen [29] observes builders coordinating design and site work by acting on the physical evidence of work already accomplished by others, rather than through direct instruction.

Where the same practice has been measured, in software, its informal human form shows clear shortcomings. Masood, Hoda, and Blincoe [30, 31] find that self-assignment in agile teams is driven by informal, individually varying preferences, task-based, developer-based, and opinion-based; where an agent's own capability is considered at all, it enters as a self-assessed and inconsistently applied factor rather than a structured record of demonstrated performance on comparable prior work. Estimation is no more reliable: developer inexperience is among the commonly reported causes of inaccurate estimates Conoscenti et al. [32], and structured reflection on one's own completed tasks does not improve accuracy, with gains coming only from external feedback rather than self-review [33]. SPD addresses these shortcomings for the structured-project domain the construction evidence points to: it replaces self-assessed capability with the trace's record of demonstrated performance, and subjective estimates with forecasts measured from it.

Experience-based ranking within eligibility. Within the pool of agents eligible for a step, meaning those meeting its minimum skillset or role requirement, selection is further informed by specific prior experience with the step's particular attributes. Two agents may share the same skillset yet differ in their recorded history against a given descriptor, one having completed several prior instances with a specific specialism, the other none. The trace is queried for each eligible agent's history of similar prior steps, and this is surfaced to inform self-selection, rather than used to centrally assign the step. This resembles history-based task assignment in automated bug triage [34], where candidates within a pre-qualified pool are ranked not by declared expertise but by their record of resolving previous instances with matching attributes.

3. The model

3.1 Definitions

Master specification. A rooted tree that breaks the product down into all its possible elements. The tree is hierarchical: the upper levels hold the high-level components, those delivered by whole systems, such as a building's foundation or its structural walls; the next levels hold the systems that can deliver each component, a foundation, for instance, by a concrete slab or by blockwork; and lower levels break those down as far as needed. Write it $T = (N, A)$: N the product elements and A the parent-child relations. A node may carry descriptors, as defined below.

$$T = (N,\ A) \tag{1}$$

Master process. A directed graph that breaks delivery down into all its possible steps. The nodes are the steps; the edges each carry a product passed between two steps, thereby determining their precedence in an acyclic graph. Write it $G = (V_s, E_{sp})$: V_s the steps, E_{sp} the products, a single step written s, and an edge $e = (u, v)$ carrying a product from step u to step v.

$$G = (V_s,\ E_{sp}) \qquad e = (u, v) \in E_{sp} \tag{2}$$

A second graph then nominally groups steps by job. Unlike the step graph, this one may contain cycles, representing delegation between jobs, as teams and organisations do when their responsibilities are mutually dependent. Write it $G_j = (V_j, E_{jp})$: V_j the jobs and E_{jp} the products passed between them.

$$G_j = (V_j,\ E_{jp}) \tag{3}$$

Mapping. A deterministic association of each node of the master specification with the steps in the master process it necessitates: a given product node always necessitates the same steps. It fixes which steps a node necessitates, not their order. Write it as a map $\mu : N \to 2^{V_s}$, from spec nodes to the sets of steps they necessitate.

$$\mu : N \to 2^{V_s} \tag{4}$$

Derivation. A project is derived by selecting a subset $S \subseteq N$ of the master specification and binding its descriptors to values; the corresponding subset of the master process, $\bigcup_{x \in S} \mu(x)$, follows through the mapping. Extending the project's specification during execution draws further subsets in the same way. Because the mapping is deterministic and the product flow is authored in the master, extension only ever adds steps and edges: it never revises what is already under way, so steps already executed, and the trace they have accumulated, stay valid.

$$\bigcup_{x \in S} \mu(x) \subseteq V_s \tag{5}$$

Agent. A resource that performs steps, as against the tools and materials a step also draws on. An agent takes up a step's contract by self-selection.

Trace. An accumulating record of execution data, tied to steps and covering the agents, tools, materials and outcomes involved. Forecasts and the selection stimulus are drawn from it. It persists across projects: a project's forecasts and selection draw on history accumulated before it began, and its own execution adds to that history. It is a set of attributed records, each tying the metrics $m(s) = (m_1, \ldots, m_n)$ recorded at a step s to the agent a that executed it.

$$\tau = \{\, (a,\ s,\ m(s)) \,\} \tag{6}$$

Descriptor. An attribute attached to a node of the master specification, numerical or categorical. A node carries descriptors $d = (d_1, \ldots, d_k)$; on an instance each takes a value, and the metrics at the steps the node necessitates are drawn from a distribution those values scale.

$$m(s) \sim \mathcal{D}(d) \tag{7}$$

Eligibility. An agent is eligible for a step if they hold the skillset it requires. Eligibility is necessary for an agent to self-select a step.

Contract. The bundle of work offered for self-selection: the inputs it draws from outside, the outputs it supplies, and the forecast over its steps, drawn from the trace. A contract is offered to every eligible agent and becomes binding on whichever self-selects it.

3.2 Illustration

Figure 1 draws the model in miniature: a master specification and master process, with the subset one project selects, here nodes A and B but not C, picked out through the mapping and shown solid against the faded remainder.

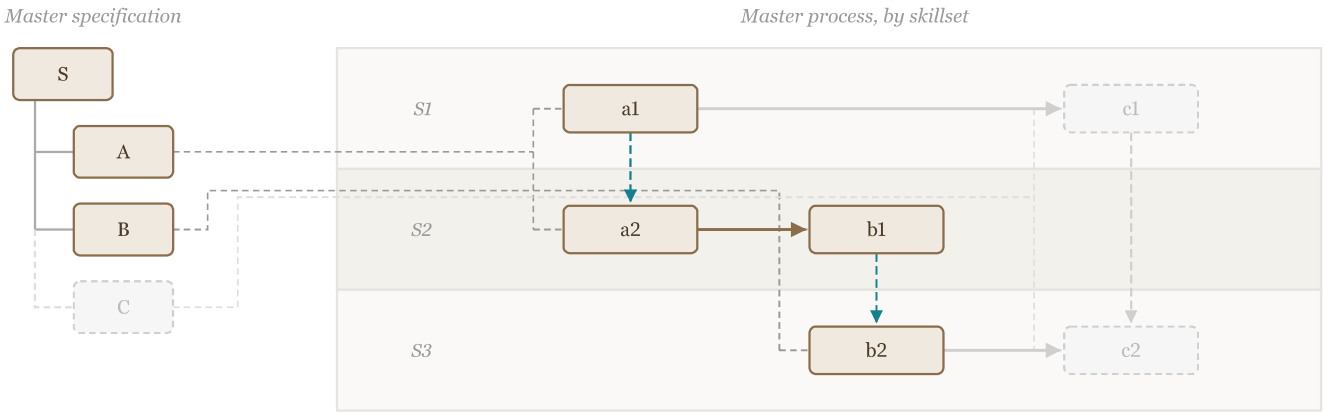


Figure 1. A master specification and master process, with the subset a project selects shown solid and the rest faded: choosing spec nodes A and B, but not C, picks out the corresponding process steps through the mapping (grey). Brown arrows carry the work in progress between a skillset's steps; teal dashed arrows carry the products passed between skillsets. A step follows the one whose output it receives.

Take a single component of that selected subset, A. It relates deterministically to the steps it necessitates, a1 and a2; a descriptor on it relates stochastically to the metrics those steps record. A descriptor is not a node, a distinction that matters for categorical ones: a node may specify a painted finish and a descriptor the colour of the paint, the first implying the painting process, the second bearing on none. Figures 2 to 4 draw these out in turn, before the worked examples below make them concrete.

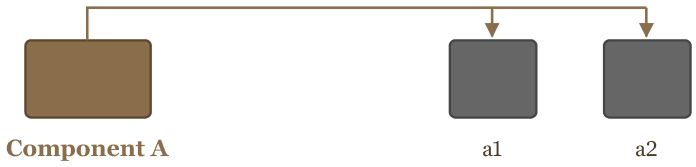


Figure 2. Component A maps deterministically to the steps it necessitates. The mapping is fixed: every instance of the specification that includes A necessitates steps a1 and a2.

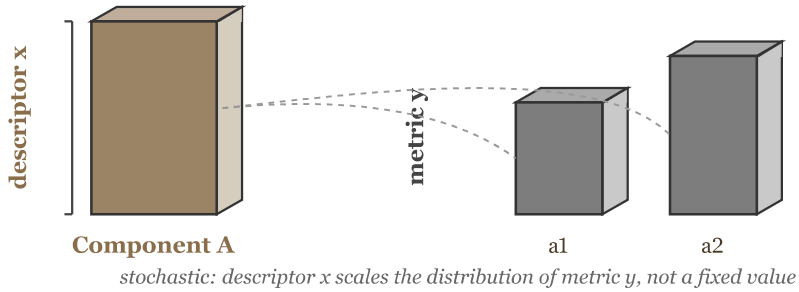


Figure 3. A descriptor on component A (descriptor x) scales the metrics recorded at each step it necessitates (metric y). The relation is stochastic: descriptor x does not fix metric y but scales the distribution metric y is drawn from.

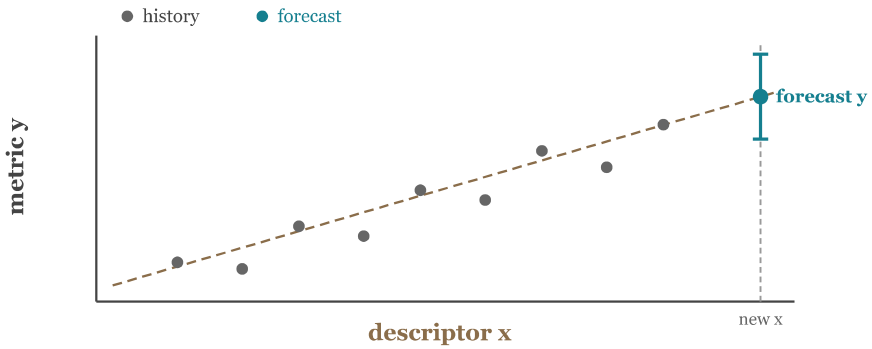


Figure 4. On a new project the descriptor x is known from the specification, but the metric y is not yet. Because metric y scales with descriptor x (Figure 3), the accumulated (descriptor x, metric y) history fits that relationship, and the forecast reads metric y at the new x off it, as a distribution with a spread rather than a fixed value.

The figures reduce the forecast to a single descriptor, a single metric, and a straight-line fit, for clarity. In practice a step's outcome typically depends on several descriptors at once, and on the agents, tools and materials involved, so the fit is multivariate. The estimator itself is not part of the model: any method that maps recorded (descriptor, outcome) tuples to a predicted distribution serves, from multivariate regression to nearest-neighbour matching or a lightweight neural network, the established approaches Section 2 surveys. What the model contributes is the structured, resource-attributed history those methods need; turning it into a forecast is a modelling choice left open.

4. Worked examples

We work two examples: a light-timber wall-frame panel and one in patient care. The second brings in three things the wall-frame example does not: extension of the specification during execution; a different kind of metric, an outcome relative to a dose of medication rather than time; and a second industry.

4.1 Wall-frame panel

The figures above are abstract; this example is concrete: a light-timber wall-frame panel. The example concerns the fabricator, who receives a set of pre-cut elements from the machinist and an assembly drawing from the designer, and is responsible for assembling the elements according to the drawing. Numbers are illustrative, chosen to exercise the arithmetic, not measured.

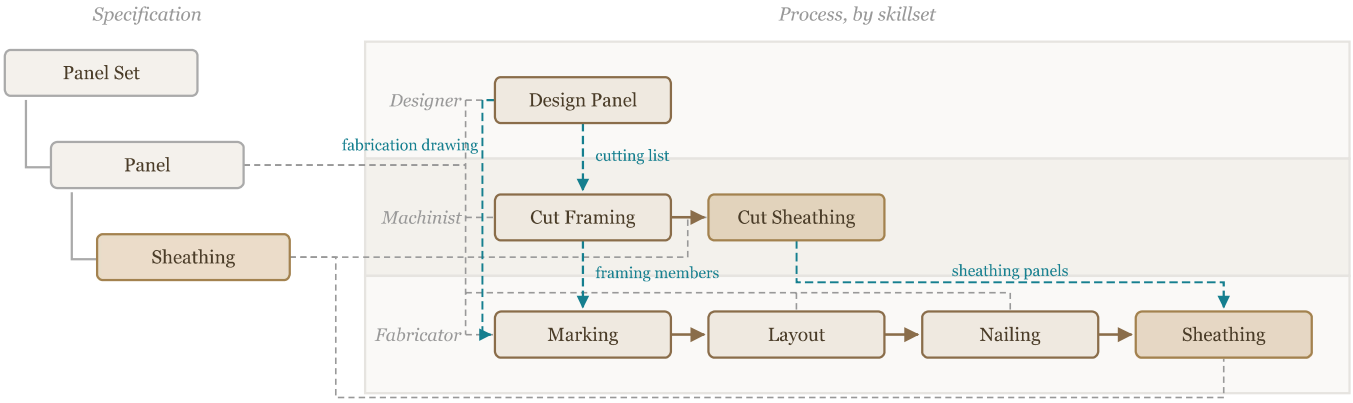
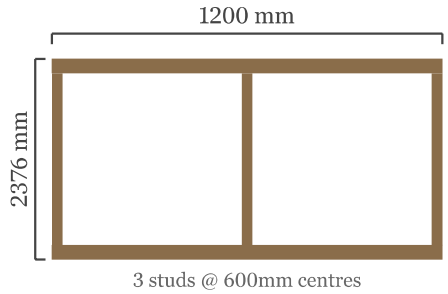


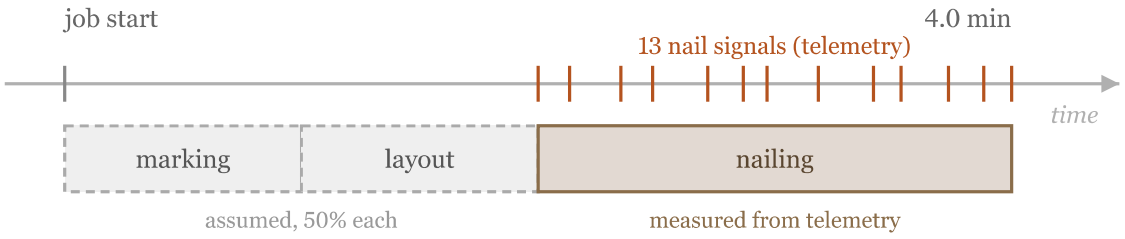
Figure 5. The sheathed panel: product tree on the left, derived process on the right in skillset lanes. Grey links are the mapping, which steps each product node necessitates (Sheathing's, tinted, appear only when it is selected). The products passed between skillsets are the designer's cutting list and fabrication drawing, the machinist's framing members and sheathing panels. The panels below zoom into the fabricator's contract.

Panel 1: the base frame



This specification resolves to 2 rails (horizontal) and 3 studs (vertical), 5 members, mapped to three fixed steps: marking, layout, nailing. Rails run the panel's width; studs are cut to wall height.

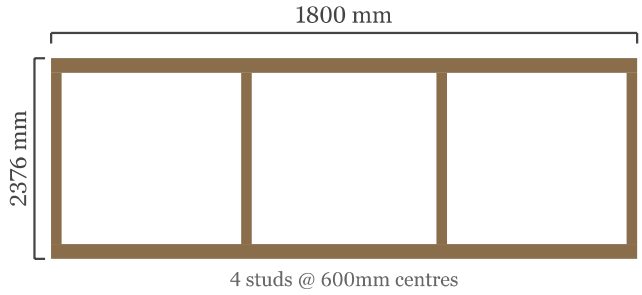
Quantity	Minimal (from spec)	Recorded
Wood	9528mm (2×1200 + 3×2376)	9710mm (+182mm waste, 1.9%)
Nails	12 (4 per stud)	13 (+1 misfire)
Time	—	4.0 min
Energy	—	0.55 kWh



Only the nailing step signals directly: telemetry records all 13 nails fired at fairly regular intervals, which bounds that step's duration. Marking and layout emit no signal, so the time before the first nail is attributed to them, split evenly for want of anything finer.

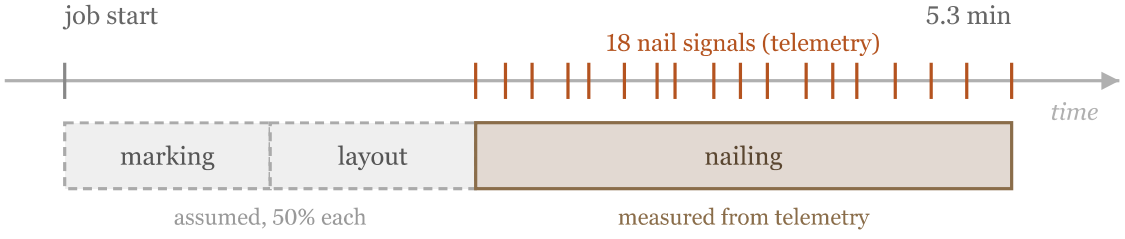
The minimal wood is fixed by the specification, and the recorded figure is what the panel consumed against it. The offcut making up the difference arises upstream at cutting rather than at assembly, so it is booked to the panel here rather than treated as the fabricator's own outcome.

Panel 2: a wider frame



Panel width is the descriptor here. At 1800mm the specification resolves to 4 studs: crossing a stud-spacing threshold adds a member, a cardinality-based extension rather than a value change on an existing component [4]. Width also scales the quantities, though not exactly, so it bears on the metric in both ways, in the member count and in what each member consumes.

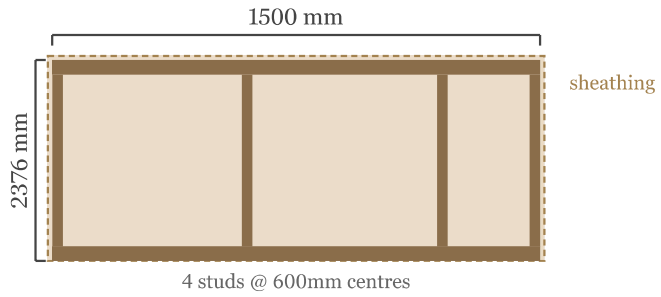
Quantity	Minimal (from spec)	Recorded	If scaled linearly from Panel 1
Wood	13104mm (2×1800 + 4×2376)	13342mm (+238mm, 1.8%)	+250mm
Nails	16 (4 per stud)	18 (+2 misfires)	+1.3
Time	—	5.3 min	5.5 min
Energy	—	0.71 kWh	0.76 kWh



Recorded values sit close to, but not exactly on, a linear scaling of Panel 1: panel width scales the metric but does not fix it, as Figure 3 shows.

Panel 3: forecasting the next frame

1500mm, 4 studs, sheathed. Sheathing is an optional feature: the mapping adds its step only where the component is selected, unlike



Panel 2's extra stud, a cardinality change to an existing step. All four steps are known from the specification with certainty; what the trace forecasts is the wastage on top of minimal quantities, and time and energy in full. Sheathing, though, has no recorded history, so unlike framing it has no trace-based forecast yet.

Wood and nail wastage are pooled across every prior frame; time and energy condition on stud count rather than width, since framing effort tracks the number of members worked rather than their length, so only Panel 2 (matching at 4 studs) is used.

Quantity	Minimal (from spec)	Forecast basis	Forecast
Wood	12504mm (2×1500 + 4×2376)	pooled waste rate across Panels 1 and 2 ≈ 1.86%	≈12737mm (+233mm); 90% within 12690–12790mm
Nails	16 (4 per stud)	pooled misfire rate across Panels 1 and 2: (1+2) / (12+16) ≈ 10.7%	≈18 (+2)
Time (framing)	—	Panel 2 only (matching stud count)	≈5.3 min
Energy (framing)	—	Panel 2 only (matching stud count)	≈0.71 kWh
Sheathing (time, energy)	—	no prior execution recorded	no forecast yet

The wood forecast is given as an interval, not a bare point: the pooled waste rate carries a spread, so the forecast is a distribution in the sense of Figure 4, and that spread is the risk it attaches. The other rows are shown as point values for brevity.

Sheathing has no trace-based forecast: the specification places the step, but the trace says nothing yet about what it costs. A descriptor-based baseline could seed a forecast before any execution, but that is outside this example.

Panel 4: the contract offered to eligible workers

Required steps and their forecasts are bundled for self-selection. Eligibility and forecast draw on different parts of the trace: eligibility on whether an agent has done a step at all; forecast on what to expect once they do.

Step	Estimate attached	Eligibility threshold
Marking	—	≥5 recorded instances
Layout	—	≥5 recorded instances
Nailing	≈12737mm wood, ≈18 nails, ≈5.3 min, ≈0.71 kWh	≥20 recorded framing instances
Sheathing	no forecast yet	≥5 recorded sheathing instances

The thresholds are illustrative. An agent eligible for sheathing on other product lines is still eligible here even without a forecast for this exact combination, as the Definitions set out.

4.2 Patient care (illustrative)

The master specification is the whole of clinical practice, rooted at the patient, whose descriptors, age, weight, ASA grade, attach to that root. A patient calls the clinic; the receptionist opens the case and, establishing that the complaint is abdominal, selects the abdominal region beneath the patient, with nothing further selected yet. That selection maps to an initial consult: the receptionist books the appointment and the GP assesses the patient (Figure 6). Almost the entire master is unselected, shown faded.

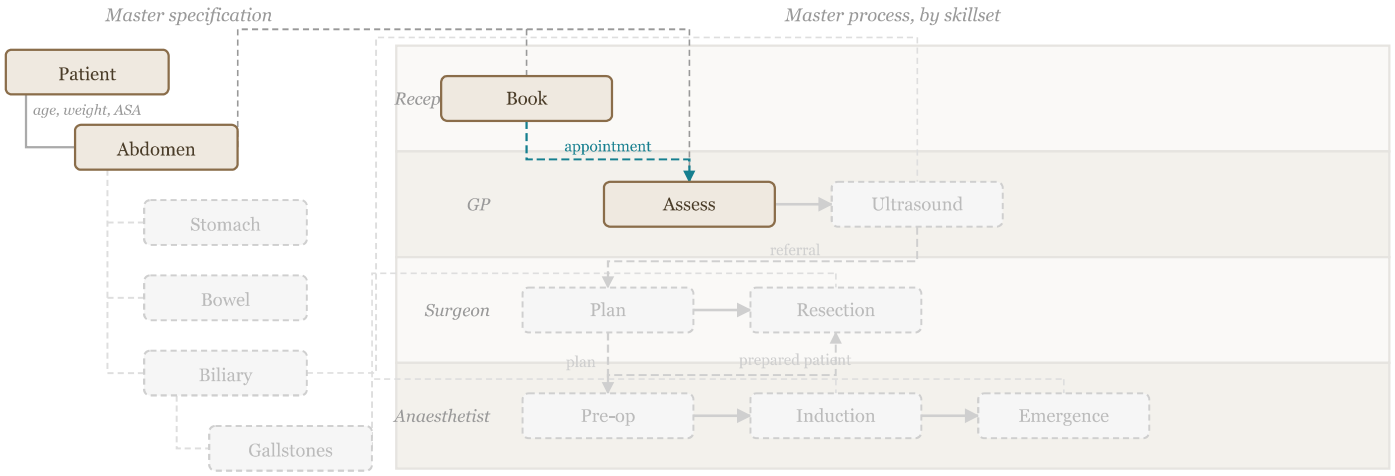


Figure 6. One master specification and master process, shown in full with unselected parts faded. The tree is rooted at the patient, which carries the descriptors; the receptionist's triage selects the abdominal region beneath it, and through the mapping (grey) that picks out the receptionist's booking and the GP's assessment, with the appointment passed between them.

Examining the patient, the GP localises the pain to the biliary system and selects that organ, one level deeper. The deeper selection necessitates a further step, an ultrasound to image the organ, and the GP's consult extends to match (Figure 7). Nothing was added to the master: the biliary node was always there; the assessment is what selected it.

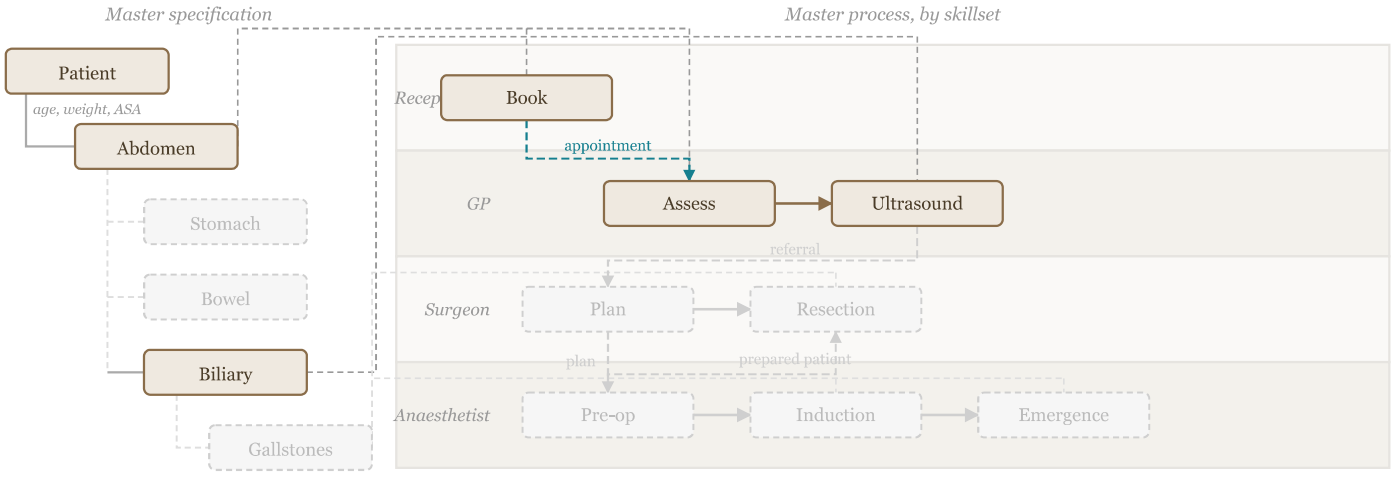


Figure 7. The GP's assessment has localised the complaint to the biliary organ, one level deeper in the same master, and selecting it necessitates the ultrasound. Only the selection, and the step it induces, are new since the previous figure.

The ultrasound identifies gallstones, selecting the issue beneath the biliary organ. That selection reaches two skillsets the consult did not, the surgeon and the anaesthetist, and the process grows to match (Figure 8). This is the model's central move, and the same rule throughout: each step's finding deepens the specification, and the deeper specification necessitates the next steps. The process progresses because the selection deepened, not because a plan was drawn; the order is inherited, so the booking, assessment and imaging already done stand.

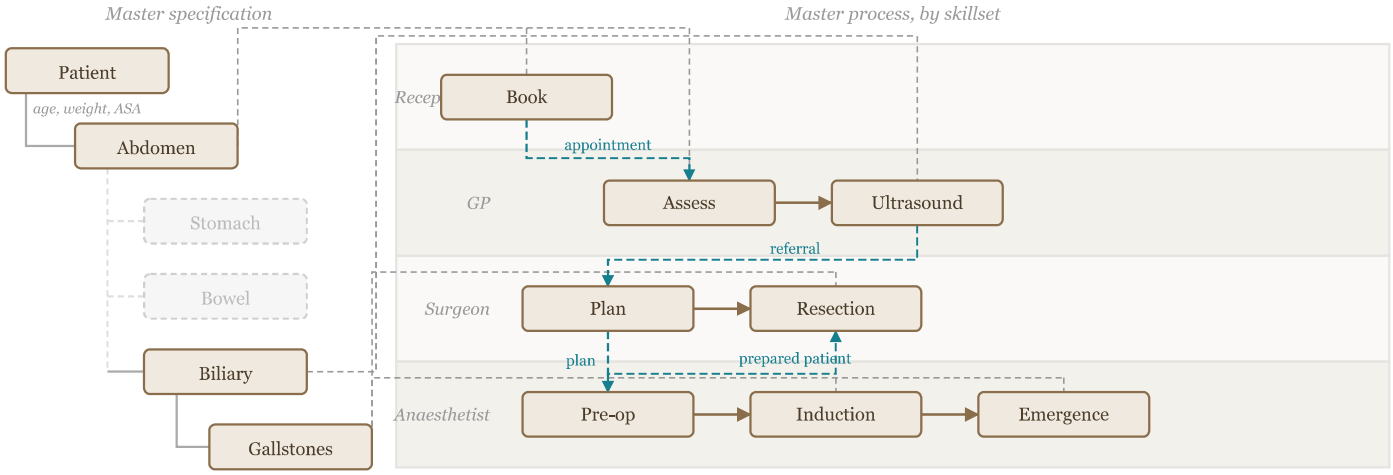


Figure 8. Imaging confirms the gallstone issue beneath the biliary organ, and selecting it reaches the surgeon and anaesthetist. The whole progression, booking to treatment, is deepening selection into one fixed master.

The induction brings in metrics the wall-frame example does not. It consumes a material, the anaesthetic drug, whose dose is recorded like any material quantity; and its outcome is measured not by time but by the largest fall in blood pressure after the drug is given. Eligibility gates the step: an anaesthetist self-selects only with a record of comparable cases, and because such cases vary widely, weight, ASA grade and the like, matching on that specific prior experience matters more here than raw case count.

Executing the step, the anaesthetist draws on that history to set the dose. For this patient's descriptors, the prior cases that match relate the dose given to the pressure drop that followed, showing which doses held blood pressure and which let it fall too far; the anaesthetist settles on a dose whose predicted drop stays within a safe range (Figure 9). As in the wall frame, the prediction is a distribution with a spread, not a single value, and that spread is the risk being weighed.

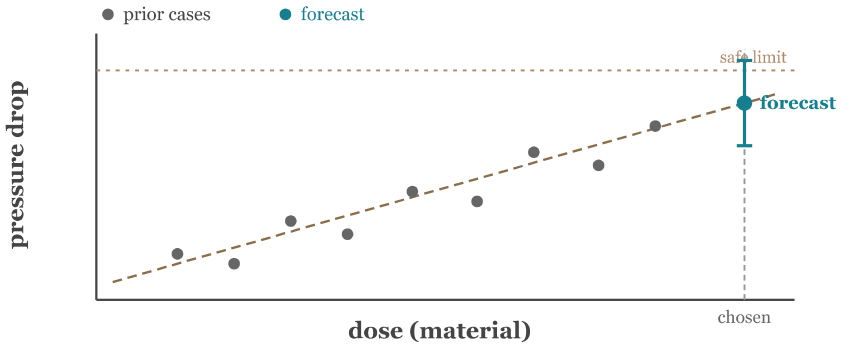


Figure 9. The anaesthetist's induction forecast, the patient-care counterpart of Panel 3. Among prior cases with matching descriptors, age, weight, ASA grade, the dose given, a recorded material, relates to the resulting fall in blood pressure; the anaesthetist reads which doses held pressure and chooses one whose predicted drop stays within a safe range, shown as a distribution with a spread.

This is where the delivered process shows itself as a live set of instructions rather than a plan fixed in advance: it updates mid-execution, reactive to the conditions found on the ground. Opening the field, the surgeon finds a stone in the common bile duct that the ultrasound missed, selecting a duct-clearance node that was always in the master but not indicated until now; the surgeon's process gains an exploration step and the anaesthetist's maintenance extends to match, mid-procedure. The update is not only structural: the newly selected node binds a descriptor, a now-compromised liver, and reading it the anaesthetist self-selects cisatracurium, which clears independently of the liver, in place of the default. The order already set stands, the steps already completed hold, and nothing returns to the GP.

5. Evaluation

The model is implemented as a working platform, on which many projects have been run against mock data sources; a full evaluation on real project data is future work.

TODO.

6. Discussion and limitations

Aggregating measured distributions along the revealed structure, as the worked example does, assumes the metrics at different steps are independent; where paths converge on a shared downstream step, that assumption does not hold, and the aggregation must account for the dependence rather than ignore it. The worked forecasts are all per-step, and none aggregates across a merge: where the process graphs do converge, at the sheathed panel's Sheathing step and the patient's Resection, the forecasts stop short of combining the incoming distributions. This case is acknowledged rather than resolved here; treating it properly is future work.

TODO. Further limitations to add.

7. Conclusion and future work

TODO.

References

[1] van der Aalst, W. M. P. (1999). On the automatic generation of workflow processes based on product structures. *Computers in Industry*, *39*(2), 97-111.

[2] Reijers, H. A., Limam, S., & van der Aalst, W. M. P. (2003). Product-based workflow design. *Journal of Management Information Systems*, *20*(1), 229-262.

[3] Kang, K. C., Cohen, S. G., Hess, J. A., Novak, W. E., & Peterson, A. S. (1990). *Feature-oriented domain analysis (FODA) feasibility study* (Technical Report CMU/SEI-90-TR-021). Software Engineering Institute, Carnegie Mellon University.

[4] Czarnecki, K., & Kim, C. H. P. (2005). Cardinality-based feature modeling and constraints: A progress report. In *International Workshop on Software Factories (OOPSLA)* (pp. 16-20).

[5] van der Aalst, W. M. P., Weske, M., & Grünbauer, D. (2005). Case handling: A new paradigm for business process support. *Data & Knowledge Engineering*, *53*(2), 129-162.

[6] Vanderfeesten, I., Reijers, H. A., & van der Aalst, W. M. P. (2007). An evaluation of case handling systems for product based workflow design. In *Proceedings of ICEIS 2007* (Vol. 3, pp. 39-46).

[7] Kelley, J. E., & Walker, M. R. (1959). Critical-path planning and scheduling. In *Proceedings of the Eastern Joint Computer Conference* (pp. 160-173).

[8] Pantelides, C. C. (1994). Unified frameworks for optimal process planning and scheduling. In *Foundations of Computer-Aided Process Operations* (pp. 253-274). CACHE Publications.

[9] Kondili, E., Pantelides, C. C., & Sargent, R. W. H. (1993). A general algorithm for short-term scheduling of batch operations - I. MILP formulation. *Computers & Chemical Engineering*, *17*(2), 211-227.

[10] Omicini, A., Ricci, A., & Viroli, M. (2008). Artifacts in the A&A meta-model for multi-agent systems. *Autonomous Agents and Multi-Agent Systems*, *17*(3), 432-456.

[11] (2018). *Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries — Part 1: Data schema (ISO 16739-1:2018)*. International Organization for Standardization.

[12] Márquez-Chamorro, A. E., Resinas, M., & Ruiz-Cortés, A. (2018). Predictive monitoring of business processes: A survey. *IEEE Transactions on Services Computing*, *11*(6), 962-977.

[13] Vanhoucke, M. (2009). *Measuring time: Improving project performance using earned value management*. Springer.

[14] Vanhoucke, M. (2012). *Project management with dynamic scheduling: Baseline scheduling, risk analysis and project control*. Springer.

[15] Vanhoucke, M. (2015). On the use of schedule risk analysis for project management. *The Journal of Modern Project Management*, *2*(3), 108-117.

[16] Malcolm, D. G., Roseboom, J. H., Clark, C. E., & Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research*, *7*(5), 646-669.

[17] Wauters, M., & Vanhoucke, M. (2016). A comparative study of artificial intelligence methods for project duration forecasting. *Expert Systems with Applications*, *46*, 249-261.

[18] Wauters, M., & Vanhoucke, M. (2017). A nearest neighbour extension to project duration forecasting with artificial intelligence. *European Journal of Operational Research*, *259*(3), 1097-1111.

[19] Flyvbjerg, B. (2006). From Nobel Prize to project management: Getting risks right. *Project Management Journal*, *37*(3), 5-15.

[20] Batselier, J., & Vanhoucke, M. (2016). Practical application and empirical evaluation of reference class forecasting for project management. *Project Management Journal*, *47*(5), 36-51.

[21] Batselier, J., & Vanhoucke, M. (2017). Improving project forecast accuracy by integrating earned value management with exponential smoothing and reference class forecasting. *International Journal of Project Management*, *35*(1), 28-43.

[22] Batselier, J., & Vanhoucke, M. (2015). Construction and evaluation framework for a real-life project database. *International Journal of Project Management*, *33*(3), 697-710.

[23] Vanhoucke, M. (2023). *The illusion of control: Project data, computer algorithms and human intuition for project management and control*. Springer.

[24] Der Kiureghian, A., & Ditlevsen, O. (2009). Aleatory or epistemic? Does it matter? *Structural Safety*, *31*(2), 105-112.

[25] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles chez Bellicositermes natalensis et Cubitermes sp. La théorie de la stigmergie: Essai d'interprétation du comportement des termites constructeurs. *Insectes Sociaux*, *6*(1), 41-80.

[26] Heylighen, F. (2016). Stigmergy as a universal coordination mechanism I: Definition and components. *Cognitive Systems Research*, *38*, 4-13.

[27] Heylighen, F. (2016). Stigmergy as a universal coordination mechanism II: Varieties and evolution. *Cognitive Systems Research*, *38*, 50-59.

[28] Smith, R. G. (1980). The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, *C-29*(12), 1104-1113.

[29] Christensen, L. R. (2013). Stigmergy in human practice: Coordination in construction work. *Cognitive Systems Research*, *21*, 40-51.

[30] Masood, Z., Hoda, R., & Blincoe, K. (2017). Motivation for self-assignment: Factors agile software developers consider. In *2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)* (pp. 92-93).

[31] Masood, Z., Hoda, R., & Blincoe, K. (2021). What drives and sustains self-assignment in agile teams. *IEEE Transactions on Software Engineering*, *48*(9), 3626-3639.

[32] Conoscenti, M., Besner, V., Vetrò, A., & Méndez Fernández, D. (2019). Combining data analytics and developers feedback for identifying reasons of inaccurate estimations in agile software development. *Journal of Systems and Software*, *156*, 126-135.

[33] Jørgensen, M., & Gruschke, T. M. (2009). The impact of lessons-learned sessions on effort estimation and uncertainty assessments. *IEEE Transactions on Software Engineering*, *35*(3), 368-383.

[34] Anvik, J., Hiew, L., & Murphy, G. C. (2006). Who should fix this bug? In *Proceedings of the 28th International Conference on Software Engineering (ICSE 2006)* (pp. 329-338).